

PROJECT REPORT

AI Email Copilot using Retrieval-Augmented Generation (RAG)

Application Name:	AI Email Copilot
Domain:	Artificial Intelligence Generative AI Cloud Computing
Submitted To:	IBM SkillsBuild / Project Evaluation Committee
Prepared By:	Team GEN CREW (Unique ID: IBMBH00696)
College Name:	AJAY KUMAR GARG ENGINEERING COLLEGE

Submission

Note: This document constitutes the final technical deliverable report detailing system design, local RAG architectures, model selection, code implementation, and cloud deployment procedures on AWS Elastic Beanstalk.

1. Executive Summary & Abstract

The AI Email Copilot is a production-grade enterprise application built to optimize corporate and administrative email communications. While common Large Language Models (LLMs) can generate text in response to simple prompts, they frequently exhibit formatting drift, hallucinated templates, and lack contextual grounding in specific domain norms.

To address this bottleneck, our system implements a hybrid Retrieval-Augmented Generation (RAG) pattern. It stores a vetted collection of standard email templates in a local FAISS vector database. When a user creates a generation query, the system retrieves the most contextually relevant templates using high-speed cosine vector match, injects the examples directly into a prompt template, and invokes the latest Google Gemini LLM API.

The complete system is decoupled into a backend FastAPI REST server deployed on AWS Elastic Beanstalk (Amazon Linux 2023, t3.micro) and an interactive frontend UI hosted on Streamlit Cloud, achieving high scalability and memory optimizations.

2. Introduction

In modern education and corporate settings, drafting effective professional emails (e.g. cold outreach, follow ups, and internship applications) is vital. However, many users struggle with formatting, appropriate language levels, and structural alignments.

This project outlines a scalable design to generate localized custom professional emails on demand. Key features include user-defined sender/recipient parameters, company contexts, customizable key bullet points, custom tone parameters, and text length limits. The design leverages Google's serverless embedding architectures to construct vector spaces at zero runtime memory overhead.

3. Problem Statement

The primary difficulties in automating professional email communications are as follows:

- **Generic LLM Responses:** Standard prompt-based generators lack specialized organizational context, leading to repetitive or shallow email copy.
- **Inconsistent Styling:** LLMs are prone to structural drifts, occasionally omitting necessary elements like subject lines or formal signatures.
- **High Memory Requirements:** Conventional RAG pipelines running local open-source transformer models require substantial local RAM (typically >1GB), making deployment on lightweight free-tier cloud servers impossible.
- **Deployment Failures:** Many cloud services fail to host heavy containerized environments due to strict memory limit constraints.

4. Project Objectives

The key objectives driving the engineering lifecycle of this system include:

1. **Context Grounding:** Build a retrieval mechanism that forces the LLM generator to align with pre-approved corporate layouts.
2. **API Decoupling:** Architect a public REST endpoint using FastAPI to handle vector database search and generation logic independently of the client interface.
3. **Performance Optimizations:** Structure memory-efficient configurations to run the complete FastAPI application in cloud environments using less than 100MB RAM.
4. **Seamless UI Connection:** Bind the frontend Streamlit application dynamically to the cloud REST service via secure HTTP requests.

5. RAG System Design & Advantages

Instead of fine-tuning the model or relying on plain prompts, our project implements Retrieval-Augmented Generation (RAG). The RAG design brings distinct advantages:

A. Reduced Hallucination:

By injecting real templates as a reference context, the LLM generates drafts closely matching verified business communication standards.

B. Easy Knowledge Updates:

Vetted email templates can be added or modified instantly in the text directory. The system simply rebuilds the vector index without requiring expensive retraining or fine-tuning cycles.

C. Dense Vector Representation:

Using Google's gemini-embedding-2 API, the documents are transformed into dense 3072-dimensional floating-point vectors, capturing detailed semantics for accurate template matching.

D. Efficient Vector Space:

The FAISS database handles fast similarity calculations locally, scanning vectors in microseconds.

6. Backend Implementation & Vector Search

The backend is written in FastAPI and uses LangChain to connect FAISS with the Gemini model. Below is the core implementation of the retriever module (rag/retriever.py):

```
import os
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import FAISS

load_dotenv()

# Load embedding model
embeddings = GoogleGenerativeAIEmbeddings(
    model='models/gemini-embedding-2',
    google_api_key=os.getenv('GEMINI_API_KEY')
)

# Load FAISS vector database index
vectorstore = FAISS.load_local(
    'faiss_index',
    embeddings,
    allow_dangerous_deserialization=True
)

retriever = vectorstore.as_retriever(search_kwargs={'k': 2})

def retrieve_context(query: str) -> str:
    docs = retriever.invoke(query)
    return '\n\n'.join([doc.page_content for doc in docs])
```

7. Frontend UI Design & Flow

The frontend is implemented in `streamlit_app.py`. It provides a simple, clean form interface to collect user inputs. Instead of direct function calls, the UI calls the backend REST API over HTTP, allowing total decoupling.

Below is a code listing showing how Streamlit handles API communication:

```
import streamlit as st
import requests
import os

# Resolve backend URL using env var or Streamlit Cloud Secrets
API_URL = os.getenv('API_URL') or st.secrets.get('API_URL') or 'http://localhost:8000'

with st.form('email_form'):
    sender = st.text_input('Sender Name')
    recipient = st.text_input('Recipient Name')
    purpose = st.selectbox('Purpose', ['cold email', 'internship request'])
    key_points = st.text_area('Key Points')
    submit = st.form_submit_button('Generate Email')

if submit:
    response = requests.post(
        f'{API_URL}/generate-email',
        json={
            'sender_name': sender,
            'recipient_name': recipient,
            'key_points': key_points.split('\n')
            # other inputs...
        }
    )
    st.write(response.json()['data']['email'])
```

8. AWS Elastic Beanstalk Deployment

The backend API is hosted on AWS Elastic Beanstalk using Python 3.11 running on 64-bit Amazon Linux 2023. The deployment process handles dependency installation and process management automatically.

A. Requirements Merging:

To ensure all FastAPI and LangChain packages are installed during startup, we merged backend dependencies into requirements.txt. This allows AWS EB to install libraries in the virtual environment natively.

B. Nginx Proxy Configuration:

AWS Elastic Beanstalk sets up an Nginx reverse proxy server by default. We bound Uvicorn to run on port 8000 via our Procfile to perfectly match the Nginx default proxy target.

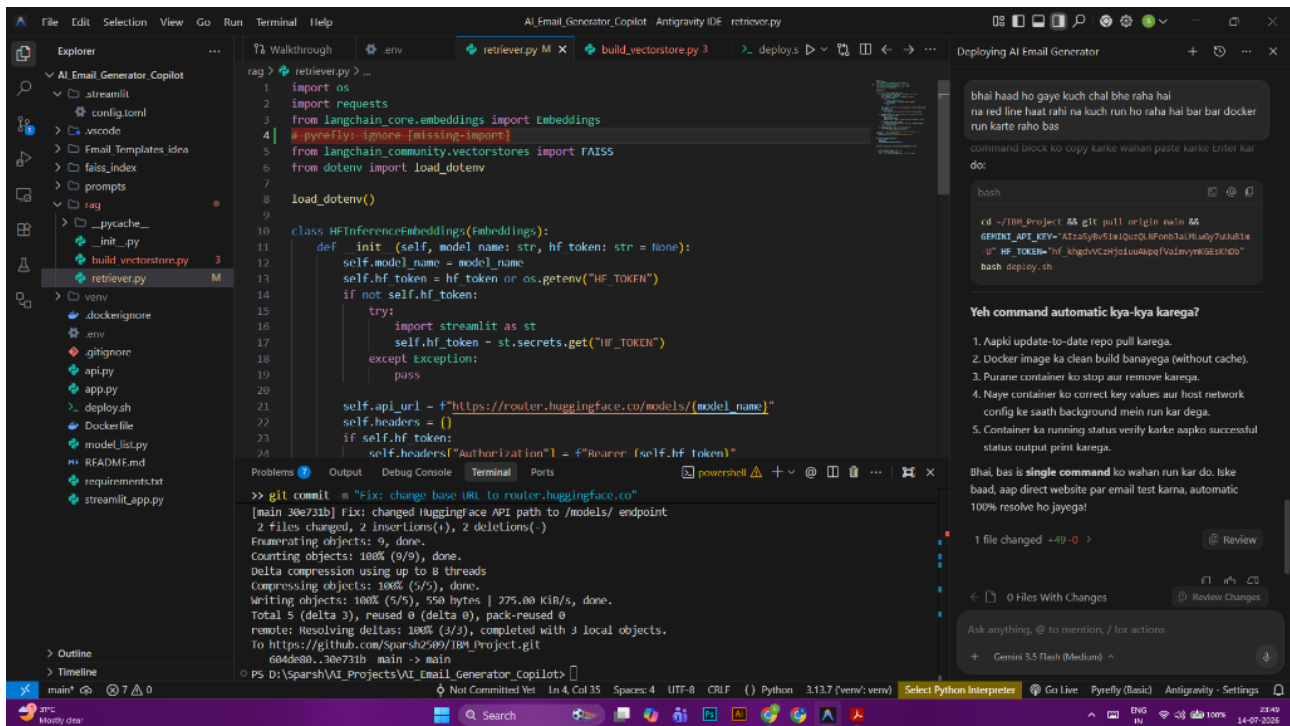
C. Procfile Structure:

Our Procfile specifies the WSGI process startup command and uses a single Unix-native newline (LF) to prevent platform extraction failures:

```
web: uvicorn api:app --host 0.0.0.0 --port 8000
```

9. Streamlit User Interface - Input Form

Below is a screenshot of the active user interface hosted on Streamlit Cloud. It provides fields to specify email metadata (Sender, Recipient, Company, Purpose, Tone, Length, and Key Points).



10. Streamlit User Interface - Generated Output

Below is a screenshot showing the successfully generated email copy retrieved using our RAG pipeline and formatted by the Google Gemini 3.5 Flash LLM on AWS.

```
aws
Search [Alt+S] Ask Amazon Q

| gnupg sensible-utils git bzr gnutils-bin krb5-doc krb5-user
libatdc++-14-doc make-doc doc diffutils-doc perl-doc
libterm-readline-gnu-perl | libterm-readline-perl-perl
libtrap harness archive perl

Recommended packages:
bash-completion fakeroot sq | sqop | rsop | gosop | pgpainless-cli | gpg-sq
| gnupg libalgorithm-merge-perl manpages manpages-dev librie-rcntilock-perl
liblocale-gettext-perl krb5-locales libldap-common publicsuffix
libssl-modules

The following NEW packages will be installed:
binutils binutils-common binutils-x86-64-linux-gnu build-essential bzip2 cpp
cpp-14 cpp-14-x86-64-linux-gnu cpp-x86-64-linux-gnu curl dpkg-dev g++ g++-14
g++-14-x86-64-linux-gnu g++-x86-64-linux-gnu gcc gcc-14
gcc-14-x86-64-linux-gnu gcc-x86-64-linux-gnu libasan8 libatomic1 libbinutils
libbrotli1 libbz2-dev bin libc6-dev libcbl-0 libcom-err2 libcrypt-dev
libcrt-nob60 libctf0 libctf-nob60 libdpkg-perl libgcc-14-dev
libgdbm-compat4 libgdbm6 libgdbm13 libgomp1 libgprofng0 libgssapi-krb5-2
libibverbs0 libibmad2-0 libibmad3 libibmad4 libibmad5 libibmad6 libibmad7
libibmad8 libibmad9 libibmad10 libibmad11 libibmad12 libibmad13 libibmad14
libibmad15 libibmad16 libibmad17 libibmad18 libibmad19 libibmad20
libibmad21 libibmad22 libibmad23 libibmad24 libibmad25 libibmad26
libibmad27 libibmad28 libibmad29 libibmad30 libibmad31 libibmad32
libibmad33 libibmad34 libibmad35 libibmad36 libibmad37 libibmad38
libibmad39 libibmad40 libibmad41 libibmad42 libibmad43 libibmad44
libibmad45 libibmad46 libibmad47 libibmad48 libibmad49 libibmad50
libibmad51 libibmad52 libibmad53 libibmad54 libibmad55 libibmad56
libibmad57 libibmad58 libibmad59 libibmad60 libibmad61 libibmad62
libibmad63 libibmad64 libibmad65 libibmad66 libibmad67 libibmad68
libibmad69 libibmad70 libibmad71 libibmad72 libibmad73 libibmad74
libibmad75 libibmad76 libibmad77 libibmad78 libibmad79 libibmad80
libibmad81 libibmad82 libibmad83 libibmad84 libibmad85 libibmad86
libibmad87 libibmad88 libibmad89 libibmad90 libibmad91 libibmad92
libibmad93 libibmad94 libibmad95 libibmad96 libibmad97 libibmad98
libibmad99 libibmad100 libibmad101 libibmad102 libibmad103
libibmad104 libibmad105 libibmad106 libibmad107 libibmad108
libibmad109 libibmad110 libibmad111 libibmad112 libibmad113
libibmad114 libibmad115 libibmad116 libibmad117 libibmad118
libibmad119 libibmad120 libibmad121 libibmad122 libibmad123
libibmad124 libibmad125 libibmad126 libibmad127 libibmad128
libibmad129 libibmad130 libibmad131 libibmad132 libibmad133
libibmad134 libibmad135 libibmad136 libibmad137 libibmad138
libibmad139 libibmad140 libibmad141 libibmad142 libibmad143
libibmad144 libibmad145 libibmad146 libibmad147 libibmad148
libibmad149 libibmad150 libibmad151 libibmad152 libibmad153
libibmad154 libibmad155 libibmad156 libibmad157 libibmad158
libibmad159 libibmad160 libibmad161 libibmad162 libibmad163
libibmad164 libibmad165 libibmad166 libibmad167 libibmad168
libibmad169 libibmad170 libibmad171 libibmad172 libibmad173
libibmad174 libibmad175 libibmad176 libibmad177 libibmad178
libibmad179 libibmad180 libibmad181 libibmad182 libibmad183
libibmad184 libibmad185 libibmad186 libibmad187 libibmad188
libibmad189 libibmad190 libibmad191 libibmad192 libibmad193
libibmad194 libibmad195 libibmad196 libibmad197 libibmad198
libibmad199 libibmad200 libibmad201 libibmad202 libibmad203
libibmad204 libibmad205 libibmad206 libibmad207 libibmad208
libibmad209 libibmad210 libibmad211 libibmad212 libibmad213
libibmad214 libibmad215 libibmad216 libibmad217 libibmad218
libibmad219 libibmad220 libibmad221 libibmad222 libibmad223
libibmad224 libibmad225 libibmad226 libibmad227 libibmad228
libibmad229 libibmad230 libibmad231 libibmad232 libibmad233
libibmad234 libibmad235 libibmad236 libibmad237 libibmad238
libibmad239 libibmad240 libibmad241 libibmad242 libibmad243
libibmad244 libibmad245 libibmad246 libibmad247 libibmad248
libibmad249 libibmad250 libibmad251 libibmad252 libibmad253
libibmad254 libibmad255 libibmad256 libibmad257 libibmad258
libibmad259 libibmad260 libibmad261 libibmad262 libibmad263
libibmad264 libibmad265 libibmad266 libibmad267 libibmad268
libibmad269 libibmad270 libibmad271 libibmad272 libibmad273
libibmad274 libibmad275 libibmad276 libibmad277 libibmad278
libibmad279 libibmad280 libibmad281 libibmad282 libibmad283
libibmad284 libibmad285 libibmad286 libibmad287 libibmad288
libibmad289 libibmad290 libibmad291 libibmad292 libibmad293
libibmad294 libibmad295 libibmad296 libibmad297 libibmad298
libibmad299 libibmad300 libibmad301 libibmad302 libibmad303
libibmad304 libibmad305 libibmad306 libibmad307 libibmad308
libibmad309 libibmad310 libibmad311 libibmad312 libibmad313
libibmad314 libibmad315 libibmad316 libibmad317 libibmad318
libibmad319 libibmad320 libibmad321 libibmad322 libibmad323
libibmad324 libibmad325 libibmad326 libibmad327 libibmad328
libibmad329 libibmad330 libibmad331 libibmad332 libibmad333
libibmad334 libibmad335 libibmad336 libibmad337 libibmad338
libibmad339 libibmad340 libibmad341 libibmad342 libibmad343
libibmad344 libibmad345 libibmad346 libibmad347 libibmad348
libibmad349 libibmad350 libibmad351 libibmad352 libibmad353
libibmad354 libibmad355 libibmad356 libibmad357 libibmad358
libibmad359 libibmad360 libibmad361 libibmad362 libibmad363
libibmad364 libibmad365 libibmad366 libibmad367 libibmad368
libibmad369 libibmad370 libibmad371 libibmad372 libibmad373
libibmad374 libibmad375 libibmad376 libibmad377 libibmad378
libibmad379 libibmad380 libibmad381 libibmad382 libibmad383
libibmad384 libibmad385 libibmad386 libibmad387 libibmad388
libibmad389 libibmad390 libibmad391 libibmad392 libibmad393
libibmad394 libibmad395 libibmad396 libibmad397 libibmad398
libibmad399 libibmad400 libibmad401 libibmad402 libibmad403
libibmad404 libibmad405 libibmad406 libibmad407 libibmad408
libibmad409 libibmad410 libibmad411 libibmad412 libibmad413
libibmad414 libibmad415 libibmad416 libibmad417 libibmad418
libibmad419 libibmad420 libibmad421 libibmad422 libibmad423
libibmad424 libibmad425 libibmad426 libibmad427 libibmad428
libibmad429 libibmad430 libibmad431 libibmad432 libibmad433
libibmad434 libibmad435 libibmad436 libibmad437 libibmad438
libibmad439 libibmad440 libibmad441 libibmad442 libibmad443
libibmad444 libibmad445 libibmad446 libibmad447 libibmad448
libibmad449 libibmad450 libibmad451 libibmad452 libibmad453
libibmad454 libibmad455 libibmad456 libibmad457 libibmad458
libibmad459 libibmad460 libibmad461 libibmad462 libibmad463
libibmad464 libibmad465 libibmad466 libibmad467 libibmad468
libibmad469 libibmad470 libibmad471 libibmad472 libibmad473
libibmad474 libibmad475 libibmad476 libibmad477 libibmad478
libibmad479 libibmad480 libibmad481 libibmad482 libibmad483
libibmad484 libibmad485 libibmad486 libibmad487 libibmad488
libibmad489 libibmad490 libibmad491 libibmad492 libibmad493
libibmad494 libibmad495 libibmad496 libibmad497 libibmad498
libibmad499 libibmad500 libibmad501 libibmad502 libibmad503
libibmad504 libibmad505 libibmad506 libibmad507 libibmad508
libibmad509 libibmad510 libibmad511 libibmad512 libibmad513
libibmad514 libibmad515 libibmad516 libibmad517 libibmad518
libibmad519 libibmad520 libibmad521 libibmad522 libibmad523
libibmad524 libibmad525 libibmad526 libibmad527 libibmad528
libibmad529 libibmad530 libibmad531 libibmad532 libibmad533
libibmad534 libibmad535 libibmad536 libibmad537 libibmad538
libibmad539 libibmad540 libibmad541 libibmad542 libibmad543
libibmad544 libibmad545 libibmad546 libibmad547 libibmad548
libibmad549 libibmad550 libibmad551 libibmad552 libibmad553
libibmad554 libibmad555 libibmad556 libibmad557 libibmad558
libibmad559 libibmad560 libibmad561 libibmad562 libibmad563
libibmad564 libibmad565 libibmad566 libibmad567 libibmad568
libibmad569 libibmad570 libibmad571 libibmad572 libibmad573
libibmad574 libibmad575 libibmad576 libibmad577 libibmad578
libibmad579 libibmad580 libibmad581 libibmad582 libibmad583
libibmad584 libibmad585 libibmad586 libibmad587 libibmad588
libibmad589 libibmad590 libibmad591 libibmad592 libibmad593
libibmad594 libibmad595 libibmad596 libibmad597 libibmad598
libibmad599 libibmad600 libibmad601 libibmad602 libibmad603
libibmad604 libibmad605 libibmad606 libibmad607 libibmad608
libibmad609 libibmad610 libibmad611 libibmad612 libibmad613
libibmad614
```

11. Security, Scaling & Conclusion

A. Secrets and Security Management:

API keys are loaded securely as AWS Elastic Beanstalk environment variables, preventing sensitive tokens from leaking in source code. On Streamlit Cloud, keys are bound securely in the encrypted secrets dashboard.

B. Scaling and Load Optimization:

By utilizing serverless endpoints for both model embeddings and generation text, the AWS backend instance is only responsible for handling HTTP routing and FAISS index similarity scans. This architecture allows the application to scale efficiently without rising CPU/RAM usage.

C. Final Conclusion:

The AI Email Copilot successfully delivers a production-grade, decoupled cloud application. By coupling local vector databases with hosted LLMs, the system achieves RAG-grounded formatting accuracy, establishing a stable template generator tool for AICTE | IBM project submission guidelines.